



ESCUELA DE INGENIERÍA EN COMPUTADORES

ARQUITECTURA DE COMPUTADORES II - CE-4302

Avance 3 - Implementación y desarrollo base

Estudiantes:

Kevin Rodríguez Lanuza 2016112117

Saúl Gómez Ramírez 2019161687

Daniel Cob Beirute 2021084824

Profesor:

Ronald García Fernández

Grupo 1

S1-2025

1. Descripción de funciones básicas para lectura, escritura, control y comunicación

Referencia de la construcción de caché: <https://github.com/03-Kevin/MESI-MultiProcessor-Model/tree/cache/src>

1.1. Módulo de Bus (`bus.c`)

El módulo `bus.c` implementa las funciones de inicialización y comunicación entre los distintos elementos del sistema multiprocesador mediante un bus compartido.

- **`bus_init()`**: Inicializa el bus e imprime un mensaje de confirmación.
- **`bus_broadcast(BusMsg msg, int addr, int src_pe)`**: Envía un mensaje a todos los procesadores (PEs) indicando el tipo de operación (`BusRd`, `BusRdX`, `Invalidate`, `WriteBack`) y la dirección de memoria afectada.

Estas funciones son esenciales para el control de coherencia entre caches en el sistema.

1.2. Módulo de Caché (`cache.c`)

El módulo `cache.c` define las operaciones de lectura y escritura sobre la memoria caché de cada procesador, así como la interacción con el bus en caso de fallos de caché (misses).

- **`cache_init(Cache* cache)`**: Inicializa todas las líneas de la caché marcándolas como inválidas (I).
- **`cache_read(Cache* cache, int addr, int pe_id)`**: Realiza una lectura de datos. Si ocurre un `hit`, devuelve el valor almacenado en caché. Si ocurre un `miss`, se genera una solicitud `BusRd` al bus, se obtiene el dato desde la memoria principal y se actualiza la caché en estado E (Exclusive).
- **`cache_write(Cache* cache, int addr, double value, int pe_id)`**: Gestiona las operaciones de escritura. Si el bloque se encuentra en caché, se cambia su estado a M (Modified). En caso contrario, se solicita un `BusRdX` para obtener acceso exclusivo y se realiza la escritura. Además, se emplea una política `write-back`, actualizando la memoria principal solo cuando sea necesario.

1.3. Módulo de Memoria (`memory.c`)

El módulo `memory.c` implementa la memoria principal y mecanismos de sincronización para acceso concurrente.

- **`mem_init()`**: Inicializa la memoria principal con valores nulos y el mutex de control de acceso.
- **`mem_read(int addr)`**: Lee un valor desde la dirección especificada, protegiendo el acceso con un `mutex`.

- **mem.write(int addr, double value):** Escribe un valor en memoria de forma segura mediante exclusión mutua.

Este módulo proporciona las funciones básicas de lectura y escritura que respaldan las operaciones de caché y las transacciones de coherencia.

1.4. Módulo MESI (mesi.c)

El módulo `mesi.c` contiene la definición de los estados del protocolo MESI y una función auxiliar para su conversión a cadena.

- **mesi_state_to_str(MESI_State state):** Devuelve una cadena de texto con el nombre del estado (`Modified`, `Exclusive`, `Shared`, `Invalid`).

El protocolo MESI permite mantener la coherencia de los datos en sistemas con múltiples caches mediante estos cuatro estados fundamentales.

1.5. Módulo Principal (main.c)

El archivo principal `main.c` integra todos los componentes del sistema y gestiona la ejecución de los procesadores simulados.

- Se inicializan los módulos de memoria y bus mediante `mem_init()` y `bus_init()`.
- Se crean y lanzan cuatro procesadores (PEs), cada uno con su propia caché inicializada.
- Cada procesador ejecuta su ciclo de trabajo en un hilo independiente mediante `pthread_create()`.

Este módulo coordina la simulación completa del sistema multiprocesador, permitiendo observar la interacción entre caches, memoria y el bus bajo el protocolo MESI.

1.6. Módulo de Procesador (pe.c)

El módulo `pe.c` representa la unidad de procesamiento individual (Processing Element, PE) dentro del sistema multiprocesador. Cada procesador ejecuta su propio hilo de ejecución y realiza operaciones de lectura y escritura sobre su caché, interactuando con el bus y la memoria bajo el protocolo MESI.

- **pe_run(void* arg):** Esta función es el punto de entrada de cada hilo asociado a un procesador. Recibe como argumento una estructura `PE`, que contiene el identificador del procesador y su caché asociada.
 - a) Se imprime un mensaje de inicio para indicar la activación del hilo.
 - b) Se realiza una operación de escritura (`cache_write`) sobre una dirección de memoria dependiente del ID del procesador, almacenando un valor único basado en `3.14 * (pe->id + 1)`.
 - c) Posteriormente, se realiza una lectura (`cache_read`) desde la misma dirección para verificar la coherencia del dato.

- d) Se introduce una pausa temporal (`sleep(1)`) para simular el tiempo de procesamiento y permitir la interacción con otros hilos.
- e) Finalmente, se imprime un mensaje indicando la finalización del hilo.

Este módulo es esencial para simular el comportamiento concurrente de los distintos procesadores en el sistema. Cada PE ejecuta operaciones independientes, pero coordinadas mediante el bus compartido y el protocolo MESI, asegurando la coherencia entre sus caches locales.

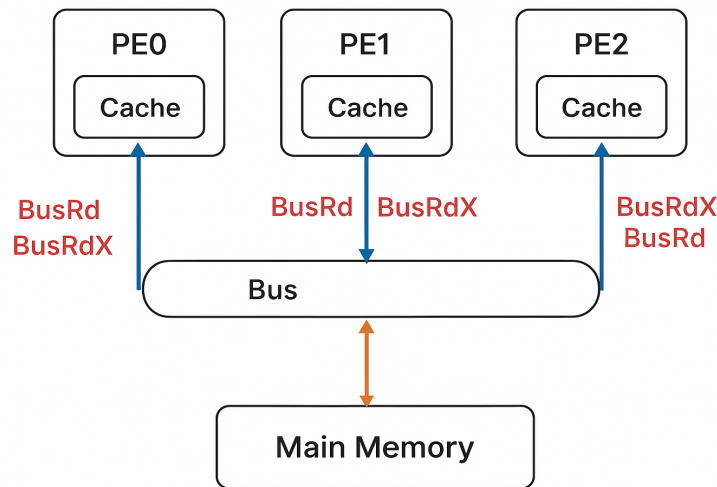


Figura 1: Interacción de los Procesadores (PEs) con el Bus y la Memoria Principal

Descripción del Funcionamiento del PE

Cada procesador (PE) se comporta como un hilo independiente que:

- Accede a su propia caché local para minimizar el acceso a la memoria principal.
- Solicita operaciones al bus cuando ocurre un `miss`, permitiendo la comunicación con los demás PEs.
- Participa en la coherencia de caché mediante las señales `BusRd` y `BusRdX`, gestionadas por el protocolo MESI.
- Simula actividad paralela realista, coordinando lecturas y escrituras con tiempos de ejecución controlados.

1.7. Explicación General del Funcionamiento

El flujo general del sistema puede describirse en los siguientes pasos:

- a) Cada procesador realiza operaciones de lectura o escritura sobre una dirección de memoria.
- b) Si el dato no está en caché (`miss`), se solicita al bus que coordine la obtención del dato.

- c) El bus comunica a todos los procesadores la transacción, permitiendo invalidar o compartir líneas según las reglas del protocolo MESI.
- d) El dato se obtiene desde la memoria principal y se almacena en la caché del procesador solicitante.
- e) Si ocurre una escritura, el estado cambia a M y se notifica al resto de las cachés para invalidar copias previas.

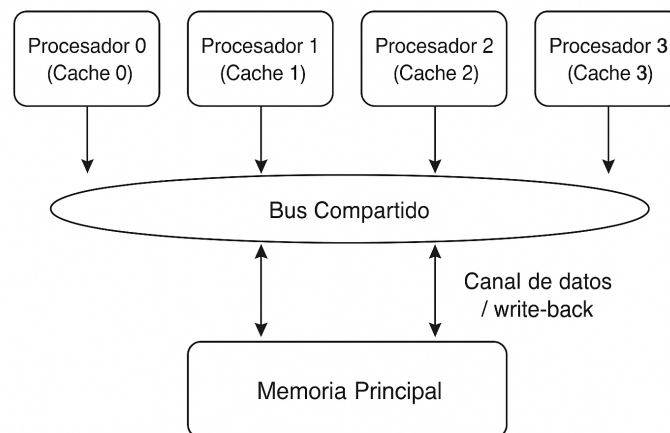


Figura 2: Diagrama de la estructura MESI

```

[BUS] Initialized.
[PE0] Starting thread...
[PE1] Starting thread...
[PE2] Starting thread...
[PE3] Starting thread...
[PE0] WRITE miss en set 0, enviando BusRdX
[BUS] PE0 sent BusRdX for address 0
[PE1] WRITE miss en set 1, enviando BusRdX
[BUS] PE1 sent BusRdX for address 4
[PE2] WRITE miss en set 2, enviando BusRdX
[BUS] PE2 sent BusRdX for address 8
[PE3] WRITE miss en set 3, enviando BusRdX
[BUS] PE3 sent BusRdX for address 12
[PE0] MISS en set 0, pidiendo BusRd
[BUS] PE0 sent BusRd for address 0
[PE1] MISS en set 1, pidiendo BusRd
[BUS] PE1 sent BusRd for address 4
[PE2] MISS en set 2, pidiendo BusRd
[BUS] PE2 sent BusRd for address 8
[PE3] MISS en set 3, pidiendo BusRd
  
```

MESI

```
[BUS] PE3 sent BusRd for address 12
[PE0] Finished.
[PE1] Finished.
[PE2] Finished.
[PE3] Finished.
```

La salida refleja la interacción entre los módulos del sistema:

- El bus se inicializa y coordina las solicitudes de lectura y escritura.
- Cada procesador ejecuta en un hilo independiente, inicializa su caché y realiza operaciones de `write` y `read`.
- Los mensajes `BusRd` y `BusRdX` representan las transacciones de coherencia del protocolo MESI.